Noosphere Backend — Architecture Document

Noosphere Backend — Architecture Document

> Generated from a full static scan of the repository. This document describes the system **as it exists in code today**, not an idealized target design.

1. Overview

Noosphere is a multi-tenant SaaS backend for clinical/behavioral-health practice management (appointments, clinical reports, billing/invoicing, payroll, custom forms, messaging, etc.). It is a **Node.js / Express 5** REST API backed by **PostgreSQL via Prisma 7**, with a separate **cron worker process** for scheduled jobs, **Socket.IO** for realtime messaging/notifications, and **AWS S3** for file storage.

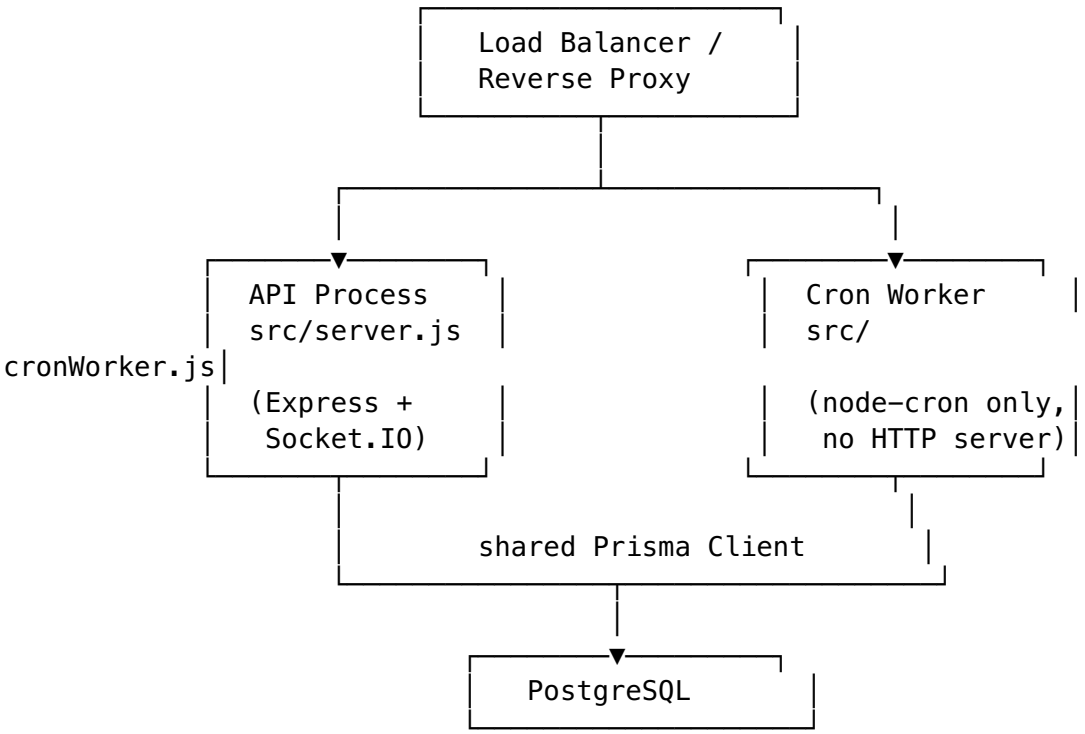
- **Runtime**: Node.js, ESM (`"type": "module"` in `package.json`)
- **Web framework**: Express 5
- **ORM**: Prisma 7 with `@prisma/adapter-pg` (driver adapter for `pg`)
- **Database**: PostgreSQL
- **Auth**: JWT (`jsonwebtoken`), `argon2`/`bcryptjs` for password hashing, Google OAuth (`passport-google-oauth20`, currently disabled), TOTP 2FA (`speakeasy`, `qrcode`)
- **Realtime**: `socket.io`
- **File storage**: AWS S3 (`@aws-sdk/client-s3`, `multer-s3`)
- **Payments**: Stripe
- **Email**: Nodemailer (SMTP) and AWS SES (`@aws-sdk/client-sesv2`)
- **Docs**: `swagger-jsdoc` + `swagger-ui-express`, generated from JSDoc annotations in route files, served at `/api-docs`
- **Scheduling**: `node-cron`, run inside a dedicated worker process (`src/cronWorker.js`)
- **Monitoring**: AWS CloudWatch custom metrics (`src/utilities/cloudWatch.js`)

Entry points (`package.json` scripts):

Script	Command	Purpose
`server` / `start`	`node(mon) src/server.js`	HTTP API + WebSocket server

Script	Command	Purpose
`worker` / `start:worker`	`node(mon) src/cronWorker.js`	Cron worker (scheduled jobs only, no HTTP)
`postinstall`	`prisma generate`	Regenerate Prisma client on install
`deploy`	`prisma generate && migrate deploy && db seed`	Production deploy pipeline

2. High-Level Architecture




```
  Cross-cutting external services (used from either/both
processes):
  - AWS S3 (file uploads/downloads, presigned URLs)
  - AWS SES / SMTP (Nodemailer) for transactional email
  - Stripe (billing, webhooks)
  - AWS CloudWatch (custom metrics)
```

The API process and the cron worker are **separate deployable processes** sharing the same codebase and Prisma schema, but each instantiates its own `PrismaService` singleton. The cron worker never calls `socketService.init()` (no HTTP server to bind Socket.IO to), so any realtime notification emitted from a cron job degrades gracefully to a console warning while the underlying data is still persisted (see `SocketService.emitToUser`).

3. Directory Layout

```
noosphere_backend/
├── prisma/
│   ├── schema.prisma          # Single schema file, ~90 models /
18 enums
│   ├── migrations/            # ~25 timestamped migration folders
│   └── seed.js                # `prisma db seed` entry point
├── src/                       # ACTIVE application code (imported
by server.js / cronWorker.js)
│   ├── server.js              # Express app bootstrap (class App)
│   ├── cronWorker.js           # Cron worker bootstrap
│   ├── health.route.js         # /health liveness endpoint
│   ├── config/                 # Cross-cutting configuration
singletons
│   ├── middleware/              # Express middleware (auth,
errors, rate limiting, audit log)
│   ├── utilities/               # Stateless/singleton helper
services (S3, mail, PDF, tokens...)
│   ├── cron/
│   │   ├── scheduler.js         # Registers all node-cron tasks
│   │   └── jobs/                # One class per scheduled job
│   └── features/               # Business domains (see §4 —
vertical slices)
├── src2/                      # ⚠  STALE DUPLICATE — see §9, not
imported anywhere, not used by any script
├── generated/                  # (empty at time of scan)
├── scripts/                    # (empty at time of scan)
├── mail.html, mailHeader.png, Welcoming.html, Logowrap.png,
logo.png  # static email/branding assets at repo root
└── src/templates/emails/        # HTML email templates used by
templateRenderer.js
```

---

4. Feature-Module Pattern (Layered / “Clean Architecture” per domain)

Every business domain lives under `src/features/<name>/` and follows the same four-layer structure. This is the core architectural convention of the codebase — **new features should follow it exactly**.

```
src/features/<feature>/
├── domain/            # Plain JS classes: constructor maps raw
fields, getters return
│                       # shaped payloads for specific use cases
(e.g. `createInvoice`,
│                       # `createSingleInvoiceOutput`). No I/O, no
framework deps.
├── infrastructure/     # Repository classes wrapping a single
Prisma model delegate.
│                       # Constructor takes `model` (e.g.
```

```
`prisma.invoice`) — thin
│                        # CRUD + a few domain-specific query
methods (pagination,
│                        # aggregation, joins via `include`).
├── application/         # Service classes containing business
logic. Constructor takes
│                        # an object of injected repositories/
services (manual DI, no
│                        # container/framework). Orchestrates
repositories, domain
│                        # objects, and utilities (email, tokens,
PDF generation).
└── presentation/
    ├── controllers/     # Wraps service calls in `express-async-
handler`. Constructor
    │                     # wires up `prismaService.getClient()` →
repositories →
    │                     # service (DI happens here, per-request-
class instance,
    │                     # not per-request).
    ├── dto/              # `Joi`-based request validators,
exported as Express
    │                     # middleware functions (`validate.js`
wraps Joi schemas).
    └── routes/           # Express Router classes. Wires path +
HTTP verb + auth
                          # middleware + DTO validator +
controller method. Also
                          # carries the Swagger/OpenAPI JSDoc
annotations inline.
```

Dependency injection pattern

There is **no DI container**. Wiring is done manually, once per controller class, in the controller's constructor:

```
// invoiceController.js
class InvoiceController {
    constructor() {
        this.prisma = prismaService.getClient();
        this.invoiceRepository = new
        InvoiceRepository(this.prisma.invoice);
        this.planRepository = new
        PlanRepository(this.prisma.billingPlan);
        this.tenantRepository = new
        TenantRepository(this.prisma.tenant);
        this.service = new InvoiceService({
            invoiceRepository: this.invoiceRepository,
            planRepository: this.planRepository,
            tenantRepository: this.tenantRepository,
        });
    }
```

```
    createInvoice = expressAsyncHandler(async (req, res) =>
        { ... });
}
```

Routes then instantiate the controller once (`this.controller = new InvoiceController()`), so a new set of repositories/service is created per **route-class** (module load), not per-request. This is fine because the underlying `prisma` client is itself a shared singleton (`prismaService`).

28 feature domains discovered

`admin`, `appointment`, `auth`, `billing`, `client`, `clinicalReport`, `customForms`, `departmentAndTeams`, `folder`, `images`, `invoice`, `issue`, `logs`, `messaging`, `notifications`, `organization`, `organizationStaff`, `performance`, `pipeline`, `planAndFeature`, `program`, `prospect`, `role`, `session`, `tenant`, `tenantBilling`, `tenantPayroll`.

Representative sub-domains per feature (not exhaustive): `client` contains client CRUD, client documents, requested documents, client authorization/authorization-services, client notification settings; `organization` contains org info/documents/licenses/diagnosis codes/session types/availability; `tenantBilling` contains service codes, rounding rules, insurance types, payers, payer-service-codes; `tenantPayroll` contains compensation types, income items, deductions, payroll cycles (+ per-staff line items).

---

5. HTTP API Layer

5.1 Bootstrap (`src/server.js`)

A single `App` class does, in order:

1. `initializeDatabase()` — connects Prisma (`prismaService.connect()`)
2. `initializeMiddlewares()`:
   - `trust proxy` set from `TRUST_PROXY_HOPS` (defaults to 1) — required for correct `req.ip` behind nginx/ALB, used by rate limiting and audit logs
   - `morgan("dev")` request logging
   - `cors()` with a custom origin-validator allowing `*.noospherehub.net`, `*.noospherehub.org`, `localhost`/`127.0.0.1` (any port), and EC2 public DNS names
   - `express.json({ limit: "50mb" })`, but **only** applied when `req.is("application/json")` and the URL isn't the Stripe webhook path (Stripe webhooks need the raw body for signature verification; multipart/form-data uploads must also skip JSON parsing)
   - `express.urlencoded({ extended: true, limit: "50mb" })`
3. `initializeRequestTracking()` — an ad-hoc middleware (not the audit logger) that records **every** request/response (method, endpoint, status, duration, IP, user-agent, and identity if authenticated) into the `ServerRequest` table, skipping `/api-docs` and `/health`
4. `initializeAuditLogging()` — `auditRequestLogger` middleware (see §7)
5. `initializeSwagger()` — mounts Swagger UI at `/api-docs`

6. `initializeRoutes()` — mounts ~65 versioned route groups under `/api/v1/*` (see §5.3)
7. `initializeErrorHandler()` — centralized error middleware (see §6)

`start()` calls `server.listen()` then initializes Socket.IO on the same HTTP server.

Global process guards: `unhandledRejection` is logged (not fatal); `uncaughtException` logs and calls `process.exit(1)`.

5.2 Routing conventions

- All API routes are versioned under `/api/v1/...`.
- Route path segments generally match feature/sub-resource names, e.g. `/api/v1/client-documents`, `/api/v1/organization/license`, `/api/v1/tenant-payroll` equivalents (`/payroll-cycles`, `/income-items`, `/deductions`, `/compensation-types`).
- `/health` is unauthenticated and excluded from request tracking/audit logging.
- Route classes self-document via inline `@swagger` JSDoc blocks, scanned by `swagger-jsdoc` from `features/**/presentation/routes/*.js`.

5.3 Authentication & Authorization

Three independent identity types, each with its own Express middleware factory in `src/middleware/auth_handlers.js`, all sharing one JWT secret (`ACCESS_TOKEN_SECRET`) and one `Bearer <token>` header convention:

Middleware	Table	Notes
`adminProtect({ superAdmin })`	`Admin`	Optional `superAdmin` flag to require `admin.superAdmin === true`; loads `roles` (name, `dataAccessLevel`, `systemModule`)
`staffProtect()`	`TenantStaff`	Also validates the **parent tenant** is active/not-deleted; blocks with `staff.tenant.suspensionReason` if the tenant was suspended; returns 403 if `tenant.featuresDisabled`
`clientProtect()`	`ClientTenant`	Client is scoped to a specific tenant relationship (a client can belong to multiple tenants via `ClientTenant`)

All three populate `req.user = { id, type: "ADMIN"|"STAFF"|"CLIENT", tenantId?, role? }` in a normalized shape consumed by the request-tracking and audit-logging middleware, plus `req.admin` / `req.tenantStaff` / `req.clientTenant` with the raw record.

Authorization beyond authentication (RBAC) is modeled via `Role` → `RoleModuleAccess`, with `dataAccessLevel` (`GLOBAL` / `INDIVIDUAL` / `TEAM`) and `systemModule`/`FeatureModule` enums, but role-based route guarding is enforced inside controllers/services rather than as reusable middleware (not centrally verified in this scan).

Payment-link & webhook access (`billing`, `invoice`) is intentionally unauthenticated (token-in-URL or Stripe-signature-verified), and is instead rate-limited via `src/middleware/rate_limit.js` (`paymentLinkRateLimiter`, `authRateLimiter`, `webhookRateLimiter`, all built on `express-rate-limit`).

---

6. Error Handling

`src/middleware/error-handler.js` exposes a single `handleError(err, req, res, next)` mounted last in the middleware chain. It pattern-matches on error shape rather than using custom error classes:

- `string` errors → 404 if the string ends in "not found" (case-insensitive), else 400
- `err.name === "ValidationError"` → 400
- JWT errors (`TokenExpiredError`, `JsonWebTokenError`, `NotBeforeError`) → 401
- If `res.statusCode` was already set to 401/403 by auth middleware before calling `next(err)`, that status is preserved
- Message prefix convention: strings starting with `"Not Authorized"` → 401, `"Forbidden"` → 403
- Default → 500

Convention for application code: services/repositories mostly `throw new Error("<message>")` and rely on this handler's string/prefix sniffing rather than typed exceptions. There is no centralized custom `AppError`/`HttpError` class.

---

7. Cross-Cutting Middleware

File	Responsibility
`auth_handlers.js`	JWT auth for Admin/Staff/Client (see §5.3)
`error-handler.js`	Centralized error → HTTP response mapping (§6)
`rate_limit.js`	`express-rate-limit` limiters for unauthenticated/sensitive endpoints (login, payment links, webhooks)
`audit-request-logger.js`	For `POST`/`PATCH` requests, on `res.finish`, derives a human-readable feature name from the URL path segment and delegates to `AuditLogger` (`features/logs/application/auditLogger.js`) to persist a business-level audit trail (tenant/client/admin id, module, feature, action, outcome). Controllers can opt out by setting `req.auditLogged = true` if they already log a more specific audit entry themselves. This is distinct from

File	Responsibility
	the raw `ServerRequest` tracking done inline in `server.js` (that captures *every* request for ops/perf; this captures *mutating* requests for compliance/ audit).

8. Data Layer

8.1 Prisma setup (`src/config/prisma.js`)

- Uses the new Prisma **driver adapter** model (`@prisma/adapter-pg` `PrismaPg`) rather than Prisma's built-in connection pooling — instantiated with `DATABASE_URL`.
- Wraps the client with `$extends` to inject a default `orderBy: { createdAt: "desc" }` on `findMany` calls for a fixed allow-list of ~50 models that have a `createdAt` column (`createdAtModels` Set) — this means **callers get newest-first by default** unless they pass their own `orderBy`.
- Exposed as a singleton (`export default new PrismaService()`), with `connect()`/`disconnect()`/`getClient()`. Both `server.js` and `cronWorker.js` import and connect this same singleton independently (separate processes → separate DB connections).

8.2 Schema scale

- **1 datasource** (PostgreSQL), **1 generator** (`prisma-client-js`)
- **~90 models**, **18 enums** (`Module`, `SubscriptionStatus`, `PlanType`, `InvoiceStatus`, `incoiceBillingFrequency` [sic — typo present in schema], `DomainType`, `DocumentRequestStatus`, `WeekDay`, `ApprovalStatus`, `ClinicalReportStatus`, `TokenOwnerType`, `ClientFormStatus`, `FeatureModule`, `PaymentSchedule`, `DataAccessLevel`, `UserType`, `NotificationEntityType`, `RescheduleRequestStatus`)
- IDs are UUID strings (`@id @default(uuid())`) throughout; timestamps use `createdAt`/`updatedAt` (`@default(now())` / `@updatedAt`)
- ~25 chronologically named migrations under `prisma/migrations/`, several with duplicate-looking names (`init` appears ≥8 times) — history is timestamp-ordered but not always descriptively named
- `prisma/seed.js` is the `prisma db seed` entry point, run automatically as part of `npm run deploy`

8.3 Domain model clusters (by model prefix/relations)

- **Identity/Org**: `Admin`, `Role`, `RoleModuleAccess`, `Department`, `DepartmentMembers`, `Teams`, `TeamMembers`
- **Tenant (practice) & staff**: `Tenant`, `TenantDeactivation`, `TenantStaff`, `TenantStaffLicenses`, `TenantStaffPayroll`, `TenantStaffDocuments`, `TenantNotificationSettings`, `TenantGeneralSettings`, `TenantAdditionalSecurityQuestions`

- **Clients (patients)**: `Client`, `ClientTenant` (many-to-many client⍰tenant, the auth anchor for `clientProtect`), `ClientDocuments`, `ClientRequestedDocuments`, `ClientNotificationSettings`, `ClientAuthorization`/`ClientAuthorizationService`, `ClientFolder`/`ClientFiles`
- **Sales pipeline**: `Pipeline`, `PipelineStage`, `PipelineItem`, `PipelineDoneTask`, `PipelineSubmittedDocument`, `PipelineItemCustomTask`, `PipelineItemCustomDocument`
- **Billing/Plans/Subscriptions**: `BillingMetadata`, `Transactions`, `Subscription`, `BillingPlan`, `Feature`, `FeatureGroup`, `SuperAdminChoices`, `TenantAdminChoices`, `Payment`, `PaymentMethod`
- **Invoicing**: `Invoice`, `InvoiceManagement` (org-wide notification config, see §10.1), `InvoiceToken` (payment-link tokens), `StripeWebhookEvent`, `PaymentAndAccountAccess`
- **Support/Issues**: `Issue`, `IssueComment`
- **Clinical programs**: `Domain`, `Program`, `Target`, `ClientProgram`, `ClientTarget`, `ClientTargetDataCollection`
- **Org profile & scheduling config**: `OrganizationInformation`, `OrganizationLicenses`, `OrganizationDocuments`, `OrganizationDiagnosisCodes`, `OrganizationSessionTypes`, `SessionTypeService`, `AvailabilityDays`, `StaffAvailability`
- **Appointments**: `Appointment`, `AppointmentRescheduleRequest`, `AppointmentService`
- **Tenant billing configuration**: `ServiceCodes`, `RoundingRules`, `InsuranceType`, `Payer`, `PayerServiceCodes`
- **Payroll**: `CompensationTypes`, `IncomeItems`, `Deductions`, `PayrollCycles`, `PayrollCycleStaffs`, `PayrollCycleStaffIncomeItems`, `PayrollCycleStaffDeductions`
- **Custom forms**: `Forms`, `FormFields`, `FormResponses`, `FormResponseFields`, `ClientForm`
- **Sessions (service delivery / timesheets)**: `Session`, `SessionData`, `SessionApproval`, `TimesheetHistory`, `SessionUpdateRequest`
- **Clinical reports**: `ClinicalReportTemplates`, `ClinicalReportTemplateSection`, `ClinicalReport`, `ClinicalReportSection`, `ClinicalReportHistory`, `ClinicalReportChangeRequest`, `ClinicalReportVersion`
- **Messaging/Notifications**: `Message`, `Notification`
- **Observability**: `Logs` (business audit trail), `ServerRequest` (raw HTTP telemetry)

---

9. `src2/` — Stale Duplicate Directory (Cleanup Candidate)

The repository contains a **second, near-complete copy** of the application at `src2/`, mirroring most of `src/features/*`. Findings:

- **Not referenced anywhere**: no script in `package.json`, no import in `src/`, references `src2` at all — it is dead weight in the repo.
- **It is older/behind `src/`**: `src2` is missing `src/cron/` and `src/cronWorker.js` entirely, `src/middleware/audit-request-logger.js`, `src/features/billing/application/stripeBillingService.js`, `src/features/`

`client/application/clientNotificationEmitter.js`, `src/features/logs/application/auditLogger.js`, `src/features/clinicalReport/application/clinicalReportNotificationService.js`, `src/features/appointment/*RescheduleRequest*`, and more. Every file that exists in both trees has diverged (`diff` shows all shared files differ).
- **Recommendation**: confirm with the team that `src2/` is an abandoned scratch/backup copy, then delete it. Keeping two divergent trees for the same features is a maintenance and security-review hazard (a fix applied to `src/` can silently not exist in `src2/`, and vice versa, misleading anyone who greps the wrong tree).

---

10. Cross-Cutting Domains Worth Calling Out

10.1 Cron Worker (`src/cron/`)

`src/cron/scheduler.js` defines a `CronScheduler` class that registers 4 jobs via `node-cron`, each independently configurable via env vars (with sane defaults), all pinned to `CRON_TIMEZONE` (default `UTC`):

Job	Default schedule	Env override	Purpos
`SubscriptionInvoiceJob`	`5 0 * * *` (00:05 daily)	`SUBSCRIPTION_INVOICE_CRON`	Genera recurri invoice active subscri
`DueAppointmentNotificationJob`	`* * * * *` (every minute)	`DUE_APPOINTMENT_NOTIFICATION_CRON`	Notifie staff/cli of imm appoint
`InvoiceManagementJob`	`15 0 * * *`	`INVOICE_MANAGEMENT_CRON`	Upcom due/ove invoice emails remind schedul auto ma overdu below)
`PaymentCollectionJob`	`30 0 * * *`	`PAYMENT_COLLECTION_CRON`	Automa paymen collecti attempt

Each job class is constructor-injectable (defaults to real `prismaService`/utilities but accepts overrides — this is clearly designed for unit testing, even though no test files were found in this scan).

`InvoiceManagementJob` (the most elaborate) reads a single global `InvoiceManagement` config row and, per run: (1) emails tenants N days before due date if not already emailed, (2) emails on the due date and flips status `Upcoming →`

Due, (3) walks a configurable reminder schedule (reminderEmail[], ordered by sendOn days overdue, capped to unpaidReminderTimesBefore entries) and sends at most one reminder per matching day, optionally attaching a generated PDF (invoicePdfGenerator.js), and (4) marks invoices Overdue once markOverDue days past due. All email delivery failures are caught per-invoice so one bad tenant doesn't abort the whole batch; results are aggregated into a summary object and logged.

The cron worker process (cronWorker.js) connects Prisma, calls cronScheduler.start(), and handles SIGINT/SIGTERM to disconnect Prisma before exit — it never touches Express or Socket.IO.

10.2 Realtime & Notifications

- src/config/socket.js (SocketService, singleton) initializes Socket.IO on the same HTTP server as the API. Rooms are keyed by ${userType}_${userId} (with a legacy STAFF_${id} alias maintained alongside the current TENANT_STAFF_${id} room for backward compatibility with older clients).
- Two socket events are handled: chatMessage (persists via MessageService, then emits to the receiver's room) and sendNotification (delegates to NotificationProcessor).
- src/utilities/notificationProcessor.js is a small rules engine driven by src/config/notifications.json: given a notification type, it looks up channel config (in_app, email), optionally filters by conditions.userType, persists an in-app Notification row + emits it over the user's socket room, and/or renders + sends an email via sendTenantEmail. Email template selection is currently a hardcoded switch in buildTemplate() (welcomeTenant, paymentSuccess, default) — most real email flows instead use src/utilities/templateRenderer.js with HTML files in src/templates/emails/, so this switch looks like a partially-migrated/legacy path worth reconciling with the newer template-based emails.
- emitToUser() is the safe public API cron jobs/services should use to push realtime events — it no-ops with a console warning if Socket.IO was never initialized (i.e., when called from the cron worker process).

10.3 File Storage (src/utilities/s3.js)

Single S3Service class wrapping @aws-sdk/client-s3 + multer-s3: - getUploadMiddleware() exposes a configured multer instance (50MB limit) with a fileFilter allow-list combining MIME-type and file-extension checks (images, PDF, Office docs, CSV, MP4/MPEG) — uploaded object keys are ${Date.now()}-${originalname}. - uploadBuffer(), getPresignedUrl(), getObjectStream(), deleteObject() for server-side buffer uploads (e.g., generated invoice PDFs) and controlled read/delete access. - **Note** (relevant to prior session in this project's history): the type/MIME metadata stored on document records (e.g. ClientDocuments.documentDetails.type) is whatever the client sends in the JSON body — this service does not independently derive/validate MIME type server-side from the uploaded file or its extension, so a frontend bug sending an incorrect type field will persist uncorrected.

10.4 Email

Two parallel mail paths exist: - `src/utilities/nodemailer.js` (`MailService`) — SMTP via `nodemailer`, used by most transactional flows (invoices, payment links). - `src/utilities/ses.js` (`emailService`) — AWS SES v2 (`sendTenantEmail`), used by the notification processor. - `src/utilities/templateRenderer.js` renders HTML templates from `src/templates/emails/` with variable substitution, used by invoice/payment-link emails.

10.5 Billing / Stripe (`features/billing`)

`StripeBillingService` (only present in `src/`, not `src2/`) wraps the Stripe SDK: customer creation with a race-safe `updateMany`-guarded upsert (`ensureCustomer`), and payment settlement wrapped in a `Serializable`-isolation Prisma transaction with retry-on-conflict (`P2034`) to handle the same invoice being settled concurrently via Stripe webhook and a client-facing confirm-payment endpoint. `StripeWebhookEvent` model presumably provides idempotency bookkeeping for inbound webhooks (webhook body is deliberately excluded from the global JSON body-parser in `server.js` so Stripe's raw-body signature check works).

10.6 Monitoring/Observability

- `ServerRequest` — every HTTP request (method, endpoint, status, duration, IP, UA, identity if authenticated), recorded inline in `server.js` (excludes `/health`, `/api-docs`); queried via `/api/v1/server-requests`.
- `Logs` model + `auditLogger.js`/`audit-request-logger.js` — business-level audit trail for mutating (`POST`/`PATCH`) requests, capturing actor, module, feature, action, and outcome (`/api/v1/logs`).
- `src/utilities/cloudWatch.js` — pushes custom metrics to AWS CloudWatch (env vars: `CLOUDWATCH_USERNAME`, `RDS_INSTANCE_ID`, `RDS_TOTAL_MEMORY_GB`, `RDS_TOTAL_STORAGE_GB`, `EC2_INSTANCE_ID`, `MY_DB_INSTANCE`) — used by the `performance` feature module.
- `swagger-jsdoc` + `swagger-ui-express` serve live OpenAPI docs at `/api-docs`, generated from inline JSDoc in every route file.

---

11. Configuration & Environment

`src/config/env.js` loads `.env` via `dotenv` and is imported first in both entry points. Environment variables observed (names only; values are secrets and not reproduced here):

`AWS_ACCESS_KEY_ID`, `AWS_REGION`, `AWS_SECRET_ACCESS_KEY`, `CLIENT_URL`, `CLOUDWATCH_USERNAME`, `DATABASE_URL`, `EC2_INSTANCE_ID`, `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `JWT_EXPIRE`, `JWT_SECRET`, `MAIL_PASS`, `MAIL_USER`, `MONGO_URI` *(present but no Mongo usage found in code — likely vestigial)*, `MY_DB_INSTANCE`, `NODE_ENV`, `PORT`, `RDS_INSTANCE_ID`, `RDS_TOTAL_MEMORY_GB`, `RDS_TOTAL_STORAGE_GB`, `S3_BUCKET_NAME`, `SERVER_URL`, `SESSION_SECRET`.

Additional env vars referenced directly in code but not present in the local `.env` sample (likely set only in deployment environments): `ACCESS_TOKEN_SECRET` (falls back to a hardcoded `"secret"` — **should always be set explicitly in every real environment**), `TRUST_PROXY_HOPS`, `SUBSCRIPTION_INVOICE_CRON`, `DUE_APPOINTMENT_NOTIFICATION_CRON`, `INVOICE_MANAGEMENT_CRON`, `PAYMENT_COLLECTION_CRON`, `CRON_TIMEZONE`, `STRIPE_SECRET_KEY`, `AWS_ENDPOINT`.

---

12. Notable Architectural Risks / Observations

These are objective observations from the scan, not fixes applied — flagging for awareness:

1. **`src2/` dead duplicate tree** (§9) — unused, divergent, should be deleted or clearly archived outside `src/`.
2. **`ACCESS_TOKEN_SECRET` has an insecure fallback** (`"secret"`) in `auth_handlers.js` — if unset in any deployed environment, JWTs would be signed/verified with a publicly-known string.
3. **No typed/custom error hierarchy** — error handling relies on string message pattern-matching (`err.message.startsWith("Not Authorized")`, string endings like `"not found"`), which is fragile to message wording changes and easy to accidentally break when a service's error text changes.
4. **Client-supplied file metadata trusted as-is** — `ClientDocuments`/similar document `type` fields are not derived/validated server-side from the actual uploaded file (see §10.3); a frontend bug (already observed in a prior session) can silently persist wrong MIME types with no backend safety net.
5. **Two parallel notification/email systems** — `notificationProcessor.js`'s hardcoded template `switch` vs. `templateRenderer.js` + HTML templates used elsewhere — suggests an incomplete migration; worth consolidating.
6. **No automated test suite** — `npm test` is a stub (`echo "Error: no test specified" && exit 1`); job classes are written with dependency-injectable constructors that appear designed for unit testing, but no test files exist in the repo at the time of this scan.
7. **`incoiceBillingFrequency` enum name typo** in `schema.prisma` (should presumably be `InvoiceBillingFrequency`) — cosmetic, but renaming later requires a migration since it's a Prisma enum name.
8. **RabbitMQ config file is fully commented out** (`src/config/rabbitMQ.js`) — either a partially-removed integration or an unfinished one; not used anywhere in this scan.